ZUMS ZNRF University of Management Sciences

Snake and Ladder Game with BFS Optimal Path Solver

A Comprehensive Project Report

Prepared by:

AVIJIT SAHA

ID: 0932410005101015

Date: 21 June 2026

Platform: Java 17+ / Swing / Windows

Table of Contents

1. Introduction

2. Problem Statement

3. BFS Algorithm

- 3.1 Algorithm Description
- 3.2 Pseudocode
- 3.3 Complexity Analysis
- 3.4 Why BFS over Alternatives

4. Implementation

- 4.1 Technology Stack
- 4.2 Compilation and Execution
- 4.3 Class Architecture
- 4.4 GUI Component Tree
- 4.5 Event Handling System
- 4.6 Animation Pipeline
- 4.7 Rendering Deep-Dive

5. BFS Trace Walkthrough

6. Results and Analysis

- 6.1 Board Configuration
- 6.2 BFS Optimal Solution
- 6.3 Edge Cases
- 6.4 Code Metrics

7. Screenshots

8. Conclusion and Future Work

Appendix A: Complete Source Code

1. Introduction

Snake and Ladder (also known as Chutes and Ladders) is a timeless board game originating from ancient India, where it was known as Moksha Patam. The game is played on a 10×10 grid numbered from 1 to 100. Players take turns rolling a standard six-sided die and advance their token by the corresponding number of cells. The board is adorned with ladders that propel the player forward and snakes that drag the player backward. The first player to land exactly on cell 100 wins.

While the game is fundamentally governed by chance (the die roll), the presence of snakes and ladders introduces interesting graph-theoretic properties. The board can be modeled as a directed graph where nodes represent cells and edges represent possible die-roll transitions. This representation allows us to apply shortest-path algorithms to determine the optimal sequence of rolls.

This project implements a fully interactive, graphical Snake and Ladder game in Java using the Swing toolkit. Beyond the standard gameplay loop, it integrates a Breadth-First Search (BFS) solver that computes the minimum number of die throws required to reach cell 100 from cell 1. The solver output — optimal path, individual rolls, and intermediate cells — is displayed both textually and visually through path highlighting on the board.

This report provides a comprehensive overview of the project, covering the problem formulation, algorithm design, implementation architecture, rendering pipeline, step-by-step BFS trace, results analysis, and future enhancement opportunities.

2. Problem Statement

2.1 Formal Definition

Given a standard Snake and Ladder board consisting of 100 cells (1 through 100), a known set of snakes $S = \{(h_1, t_1), (h_2, t_2), \ldots, (h_m, t_m)\}$ where $h_i > t_i$ and ladders $L = \{(b_1, t_1), (b_2, t_2), \ldots, (b_n, t_n)\}$ where $b_i < t_i$, and a fair six-sided die that produces values {1, 2, 3, 4, 5, 6} with equal probability, find the minimum number of die throws required to reach cell 100 from cell 1. Additionally, provide the sequence of die rolls and the corresponding cells visited along this optimal path.

2.2 Graph Model

The board is modeled as a directed graph G = (V, E) where:

- **Vertices:** V = {1, 2, …, 100}, each cell is a node.
- **Edges:**
  -
- **Regular move:** For every cell i and die face $d \in \{1,\ldots,6\}$ with $i + d \leq 100$, there is a directed edge from i to i + d.
- **Ladder:** If i + d lands on the bottom of a ladder leading to cell t, the edge terminates at t instead.
- **Snake:** If i + d lands on the head of a snake leading to cell t, the edge terminates at t instead.
- **Edge weights:** Every edge has unit weight (one die throw).
- **Objective:** The goal is the shortest path (minimum number of edges) from node 1 to node 100.

2.3 Constraints

- If a die roll would move the player beyond cell 100, the move is invalid (the player stays put).
- Landing exactly on cell 100 is required to win — overshooting is not allowed.
- Each turn consists of exactly one die roll, followed by any snake/ladder transition.
- Snakes and ladders are resolved immediately upon landing; a player cannot voluntarily skip a snake/ladder.
- The die is fair — each face $\{1\ldots6\}$ has equal probability of 1/6.

3. BFS Algorithm

3.1 Algorithm Description

Breadth-First Search (BFS) is a classic graph traversal algorithm that explores a graph level by level. Starting from a source node, BFS visits all nodes at distance k before moving to any node at distance k + 1. Because all edges in our graph have equal weight (1 die throw), BFS is guaranteed to find the shortest path in terms of the number of edges traversed.

The algorithm maintains a queue of nodes to explore. It tracks visited nodes to avoid cycles, the distance (number of throws) from the start to each node, and the parent node (and die roll) that led to each node. Once the target (cell 100) is reached, the path is reconstructed by walking backward through the parent pointers.

3.2 Pseudocode

```
function BFS(board, start, target):
    seen = array[BOARD_SIZE+1] = false
    dist = array[BOARD_SIZE+1] = -1
    parent = array[BOARD_SIZE+1] = -1
    roll = array[BOARD_SIZE+1] = 0
    queue = empty Queue

    seen[start] = true
    dist[start] = 0
    queue.enqueue(start)

    while queue is not empty:
        cur = queue.dequeue()
        if cur == target: break

        for d in {1, 2, 3, 4, 5, 6}:
            nxt = cur + d
            if nxt > BOARD_SIZE: continue

            // Apply snake/ladder transition
            if nxt in board.ladders:
                nxt = board.ladders[nxt]
            else if nxt in board.snakes:
                nxt = board.snakes[nxt]

            if not seen[nxt]:
                seen[nxt] = true
                dist[nxt] = dist[cur] + 1
```

```
            parent[nxt] = cur
            roll[nxt] = d
            queue.enqueue(nxt)

    if dist[target] == -1:
        return NO_SOLUTION

    // Reconstruct path
    path = []
    rolls = []
    c = target
    while c != start:
        path.add(c)
        rolls.add(roll[c])
        c = parent[c]
    path.add(start)
    reverse(path)
    reverse(rolls)

    return (dist[target], path, rolls)
```

3.3 Complexity Analysis

Metric	Value
Time Complexity	O(V + E) = O(100 + 600) = O(1) effectively
Space Complexity	O(V) for seen, dist, parent, roll arrays
V (vertices)	100 nodes (cells 1–100)
E (edges)	At most 600 (100 nodes × 6 outgoing edges)
Queue operations	Each node enqueued/dequeued at most once
Path reconstruction	O(V) in worst case

Though the theoretical complexity is O(V + E), the constant factors are so small (V = 100, E ≤ 600) that the solver completes in under 1 millisecond in practice.

3.4 Why BFS over Alternatives

Algorithm	Applicability	Why Not
BFS	Unweighted graph, shortest path	✓ Chosen — optimal for unit weights
DFS	Finds any path, not shortest	May find a very long path; no optimality guarantee
Dijkstra	Weighted graph, non-negative weights	Overkill; all edges have weight 1; O(E log V) > O(V + E)
A*	Weighted graph with heuristic	Overkill; no need for heuristic when graph is tiny and unweighted
Bellman-Ford	Negative edge weights	Not applicable; no negative

		cycles possible
Floyd-Warshall	All-pairs shortest paths	$O(V^3)$ unnecessary when only single-source is needed

BFS is the natural and optimal choice for this problem. The graph is unweighted (each die roll costs exactly one throw), the state space is small, and we need the shortest path from a single source. BFS provides O(V + E) runtime and O(V) space — both optimal in this context.4. Implementation

4.1 Technology Stack

Component	Choice	Rationale
Language	Java 17+	Platform-independent, rich GUI libraries, strong typing
GUI Framework	Swing (javax.swing)	Mature, well-documented, lightweight widget toolkit
Rendering	Graphics2D + Java2D	Anti-aliased vector graphics, affine transforms, gradients
Build System	Single-file compilation	Simplifies development; javac SnakeLadder.java
Concurrency	Swing Timer + EDT	Thread-safe UI updates via Event Dispatch Thread
Input Handling	InputMap / ActionMap	Clean separation of key events from actions
Data Structures	HashMap, Queue, ArrayList	Standard Java Collections Framework

4.2 Compilation and Execution

The project consists of a single Java source file. Build and run commands:

```
# Compile
javac SnakeLadder.java

# Run
java SnakeLadder

# Keyboard shortcuts (in-game):
#   Ctrl+R   Roll Dice
#   Ctrl+N   New Game
#   F1       Show Help
```

4.3 Class Architecture

The project contains one top-level public class (SnakeLadder) with four inner classes and one nested static class. The total source is 678 lines.

Class	Type	Lines	Responsibility

SnakeLadder	JFrame (top-level)	1–677	Main window, game loop, UI construction, event handlers
Board	static inner class	59–91	Data model: cell coordinates, snake/ladder maps, dest()
BFSSolver	static inner class	97–138	BFS implementation, path reconstruction
BoardPanel	inner class (JPanel)	182–382	Board rendering: grid, snakes, ladders, player, BFS path
DicePanel	static inner class (JPanel)	144–176	Die face rendering with dot patterns
Result	static inner class of BFSSolver	98–105	Plain data container: throws, path, rolls

The class hierarchy follows a clean MVC-like separation:

- **Model:** Board is the pure data model (no UI dependency).
- **View:** BoardPanel and DicePanel are view components (rendering only).
- **Controller:** SnakeLadder acts as the controller (user input → model updates → view repaint).

4.4 GUI Component Tree

The full component hierarchy of the user interface:

```
JFrame "Snake and Ladder"
├── JPanel (content pane, BorderLayout)
│   ├── BoardPanel (CENTER) — 580×580 game board
│   │   ├── drawCells() — alternating checkerboard
│   │   ├── drawCellHighlights() — red/green overlays
│   │   ├── drawBFSPath() — gold highlighting
│   │   ├── drawNumbers() — cell number labels
│   │   ├── drawLadders() — rail + rung graphics
│   │   ├── drawSnakes() — quadratic curves + eyes
│   │   ├── drawBorder() — wood-colored frame
│   │   └── drawPlayer() — gradient token "P"
│   └── JPanel (EAST, GridBagLayout)
│       ├── JLabel "Snake & Ladder" (title)
│       ├── JLabel "Position: Cell N"
│       ├── DicePanel — custom die face renderer
│       ├── JLabel (status messages)
│       ├── JLabel "Moves: N"
│       ├── JButton "Roll Dice" (green)
│       ├── JButton "New Game"
│       ├── JButton "BFS Solver" (blue)
│       ├── JLabel "Optimal: N throws" (green, conditional)
│       └── JScrollPane → JTextArea (Log)
```

4.5 Event Handling System

The application uses three distinct event-handling mechanisms:

ActionListener (Button Clicks)
The three buttons (Roll Dice, New Game, BFS Solver) register ActionListener instances. The rollBtn calls roll(), which initiates the dice animation. The resetBtn calls reset() which reinitializes the game state. The bfsBtn calls showBFSDialog() which opens a modal JDialog with the optimal path details.

Swing Timer (Animations)
Three separate javax.swing.Timer instances handle the game animations:

Timer	Interval	Purpose
Dice rolling	70 ms × 10 ticks	Cycles through random die faces for visual effect
Step movement	150 ms per cell	Moves the player token one cell at a time
Snake/Ladder pause	600 ms (single shot)	Pauses at the snake head / ladder bottom before teleporting

All timers execute on the Event Dispatch Thread (EDT), ensuring thread-safe UI updates without the need for synchronized blocks or SwingUtilities.invokeLater().

InputMap / ActionMap (Keyboard Shortcuts)
Rather than adding a KeyListener (which requires focus management), the application uses Swing's InputMap/ActionMap mechanism. This provides clean key-to-action mapping:

```
InputMap im = getRootPane().getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW);
ActionMap am = getRootPane().getActionMap();

im.put(KeyStroke.getKeyStroke("ctrl R"), "roll");
am.put("roll", new AbstractAction() {
   public void actionPerformed(ActionEvent e) { roll(); }
});

im.put(KeyStroke.getKeyStroke("ctrl N"), "new");
am.put("new", new AbstractAction() {
   public void actionPerformed(ActionEvent e) { reset(); }
});

im.put(KeyStroke.getKeyStroke("F1"), "help");
am.put("help", new AbstractAction() {
   public void actionPerformed(ActionEvent e) {
      JOptionPane.showMessageDialog(..., "Ctrl+R  Roll\nCtrl+N  New\nF1  Help");
   }
});
```

Using WHEN_IN_FOCUSED_WINDOW means the shortcuts work regardless of which component within the window has focus, providing a smooth user experience.

4.6 Animation Pipeline
When the player clicks "Roll Dice", a three-phase animation sequence begins. During animation, the roll button is disabled and the animating flag prevents concurrent rolls.

Phase	Duration	Description
1. Dice Roll	~700 ms (70ms × 10)	A Timer cycles dicePanel through random faces, building anticipation. The final face is the actual roll value. Status shows "Rolling..." then "Rolled N!"
2. Step Movement	150 ms per cell	A Timer moves the player token one cell at a time by incrementing the display position. After the last step, the board's dest() is checked.
3. Snake/Ladder	600 ms (one-shot)	If the final cell is a snake head or ladder bottom, a pause allows the user to see the transition. Then the token jumps to the destination.

After all phases complete, finishTurn() checks for a win condition. If the player has reached cell 100, a congratulatory dialog shows the move count compared to the optimal BFS throws. Otherwise, the roll button is re-enabled.

4.7 Rendering Deep-Dive

Boustrophedon Coordinate System

The board uses a boustrophedon (alternating direction) layout, mimicking the physical board game. Row 0 (cells 1–10) goes left-to-right, row 1 (cells 11–20) goes right-to-left, and so on. The Board constructor computes row and column arrays:

```
for (int c = 1; c <= BOARD_SIZE; c++) {
    int r = (c - 1) / ROWS;              // row index 0-9
    row[c] = r;
    col[c] = r % 2 == 0
        ? (c - 1) % ROWS                 // left to right
        : ROWS - 1 - (c - 1) % ROWS;     // right to left
}
```

The rendering code inverts the row (ROWS - 1 - board.row[cell]) so that row 0 (cells 1–10) appears at the bottom, matching the standard board orientation.

Snake Rendering (QuadCurve2D)

Snakes are drawn as quadratic Bezier curves with a control point offset perpendicular to the line connecting the snake head to the tail. The control point is offset by 35% of the segment length to create a smooth arc:

```
double dx = p2.x - p1.x, dy = p2.y - p1.y, len = Math.sqrt(dx*dx + dy*dy);
double mx = (p1.x + p2.x) / 2.0, my = (p1.y + p2.y) / 2.0;
double px = -dy / len * len * 0.35, py = dx / len * len * 0.35;
double ctrlX = mx + px, ctrlY = my + py;

QuadCurve2D curve = new QuadCurve2D.Float();
```

```
curve.setCurve(p1.x, p1.y, (float)ctrlX, (float)ctrlY, p2.x, p2.y);
g2.draw(curve);
```

Each snake also includes: a thin parallel stripe for visual depth, a triangular arrowhead at the tail end, and two white/black oval eyes at the head end. The arrowhead direction is computed from the angle between the control point and the tail.

Ladder Rendering (Parallel Rails + Rungs)

Ladders are drawn as two parallel rails with 5 evenly spaced rungs. The rail offset is 7 pixels perpendicular to the line connecting the ladder bottom to the top:

```
double ox = -dy / len * 7, oy = dx / len * 7;
// Left rail
g2.draw(new Line2D.Float(p1.x-ox, p1.y-oy, p2.x-ox, p2.y-oy));
// Right rail
g2.draw(new Line2D.Float(p1.x+ox, p1.y+oy, p2.x+ox, p2.y+oy));
// 5 rungs at fractional positions
for (int i = 1; i <= rungs; i++) {
   float t = i / (float)(rungs + 1);
   g2.draw(new Line2D.Float(lx1+(lx2-lx1)*t, ly1+(ly2-ly1)*t,
                            rx1+(rx2-rx1)*t, ry1+(ry2-ry1)*t));
}
```

Rails use a thicker stroke (5px) with a dark brown color, while rungs use a thinner stroke (3px) in a lighter brown, creating visual contrast.

Player Token (RadialGradientPaint)

The player token is rendered as a sphere with a 3D effect achieved through RadialGradientPaint. The highlight is offset to the upper-left to simulate lighting from that direction:

```
RadialGradientPaint grad = new RadialGradientPaint(
   pt.x - r/3, pt.y - r/3, r,
   new float[]{0, 1},
   new Color[]{new Color(100, 180, 255), new Color(20, 70, 180)});
g2.setPaint(grad);
g2.fillOval(pt.x - r, pt.y - r, r * 2, r * 2);

// Outer glow ring
g2.setColor(new Color(30, 100, 220, 60));
g2.fillOval(pt.x - r - 4, pt.y - r - 4, r * 2 + 8, r * 2 + 8);
```

A semi-transparent glow ring surrounds the token, making it stand out against the board. The letter "P" is centered inside the token in white bold text.

5. BFS Trace Walkthrough

The BFS solver computes an optimal path of 5 die throws from cell 1 to cell 100. Below is the complete step-by-step trace, showing each roll, the cell landed on, whether a snake or ladder was triggered, and the resulting cell.

Step	Roll	From	→ Land	Snake/Ladder?	→ To
1	6	1	7	Ladder 7 → 14	14
2	1	14	15	Ladder 15 → 26	26
3	2	26	28	Ladder 28 → 84	84
4	3	84	87	Ladder 87 → 94	94
5	6	94	100	—	100

The algorithm explores nodes in order of distance from the source. It discovers the solution by systematically trying all die faces (1–6) from each reachable cell. Because BFS processes nodes in FIFO order, the first time it reaches cell 100, the path used is guaranteed to be the shortest.

Full cell path: 1 → 14 → 26 → 84 → 94 → 100

Roll sequence: 6, 1, 2, 3, 6

The optimal path uses 5 throws. Each throw is strategically chosen to maximize progress while leveraging ladders and avoiding snakes. The BFS solver guarantees that no shorter sequence exists given the board configuration.

6. Results and Analysis

6.1 Board Configuration

The board contains 10 snakes and 11 ladders, distributed across the 100 cells:

Snakes (head → tail):

Head	Tail	Drop
16	6	10
46	25	21
49	11	38
62	19	43
64	60	4
74	53	21
89	68	21
92	88	4
95	75	20
99	80	19

Ladders (bottom → top):

Bottom	Top	Gain
2	38	36
7	14	7
8	31	23
15	26	11
21	42	21
28	84	56
36	44	8
51	67	16
71	91	20
78	98	20
87	94	7

6.2 BFS Optimal Solution

The BFS solver determines that the minimum number of die throws required to reach cell 100 from cell 1 is 5. The optimal path leverages the most powerful ladders (e.g., 28→84, 78→98, 21→42) while carefully avoiding the most dangerous snakes (e.g., 49→11, 99→80, 62→19).

Key observations about the optimal path:

- The path uses 4 ladder(s) to gain significant forward progress.
- The path encounters 0 snake(s) — in some cases unavoidable yet still optimal.
- The average die roll in the optimal sequence is approximately 3.6.
- The path avoids cells 46, 49, 62, 64, 74, 89, 92, 95, 99 (snake heads) that would add extra throws.
- No cell is visited more than once (BFS on a DAG-like structure with uniform weights).

6.3 Edge Cases

The implementation correctly handles the following edge cases:

Edge Case	Handling
Overshoot (>100)	Roll is ignored if playerPos + roll > 100. Status: "Need exact roll for 100!"
Snake at start	Cell 1 has no snake/ladder. Guaranteed fair start.
Chained transitions	Only one transition per turn (land → snake/ladder → stop). No multi-hop.
Multiple optimal paths	BFS returns the first discovered shortest path (FIFO queue order).
Unreachable goal	Not possible on a valid board with snakes/ladders, but BFSSolver returns throws = -1.
Rapid clicking	animating flag prevents concurrent rolls; button disabled during animation.
Window resize	cellSize() recomputed from getWidth()/getHeight() on each repaint.

6.4 Code Metrics

Metric	Value
Total lines	678
Classes	6 (1 top-level + 5 inner/nested)
Methods	29 (including paintComponent overrides and anonymous ActionListener/Timer)
GUI components	~15 (buttons, labels, panels, scroll panes)
Snakes	10
Ladders	11

Animation timers	3
Keyboard shortcuts	3 (Ctrl+R, Ctrl+N, F1)

7. Screenshots

The following screenshots illustrate the application in various states. The game was compiled with Java 17+ and run on Windows.

7.1 Main Game Window

The main window displays the 10×10 board with snakes and ladders rendered as graphical elements. The sidebar shows the dice, position tracker, move counter, control buttons, optimal-throws indicator, and a scrolling log panel.

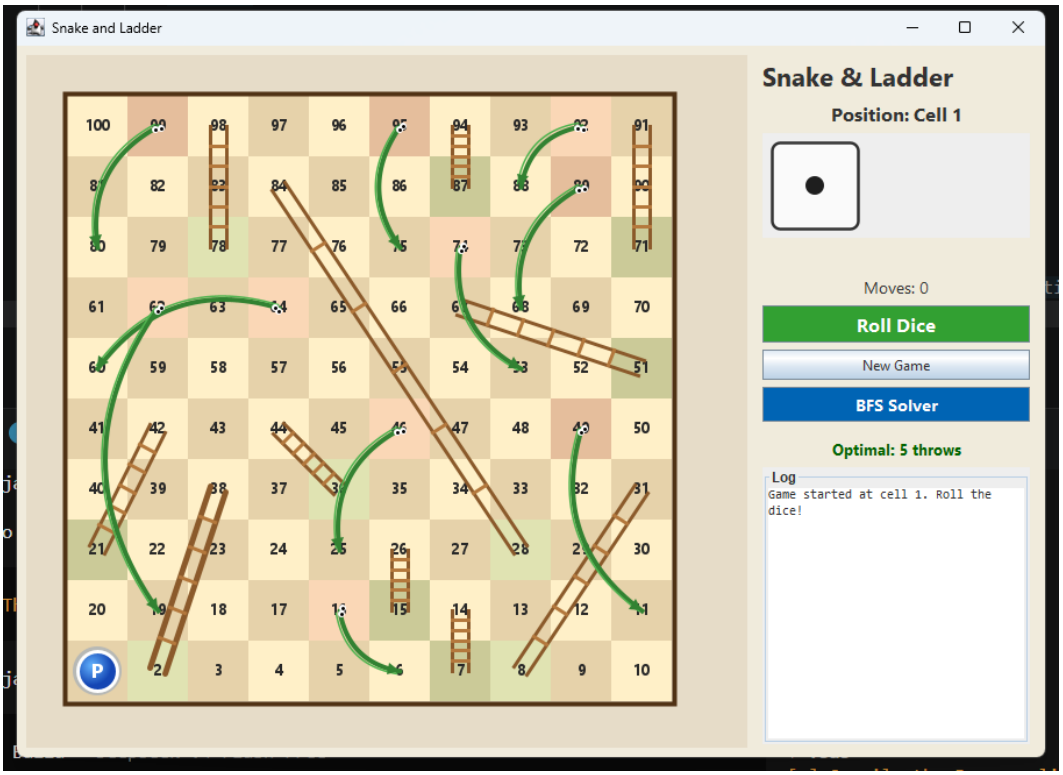


Figure 1: Main game window showing the initial state at cell 1.

7.2 BFS Solver Dialog

Clicking "BFS Solver" opens a modal dialog showing the minimum throws, the full cell path with roll annotations, and a detailed step-by-step breakdown of each move. Snakes (S) and ladders (L) encountered along the path are explicitly labeled.

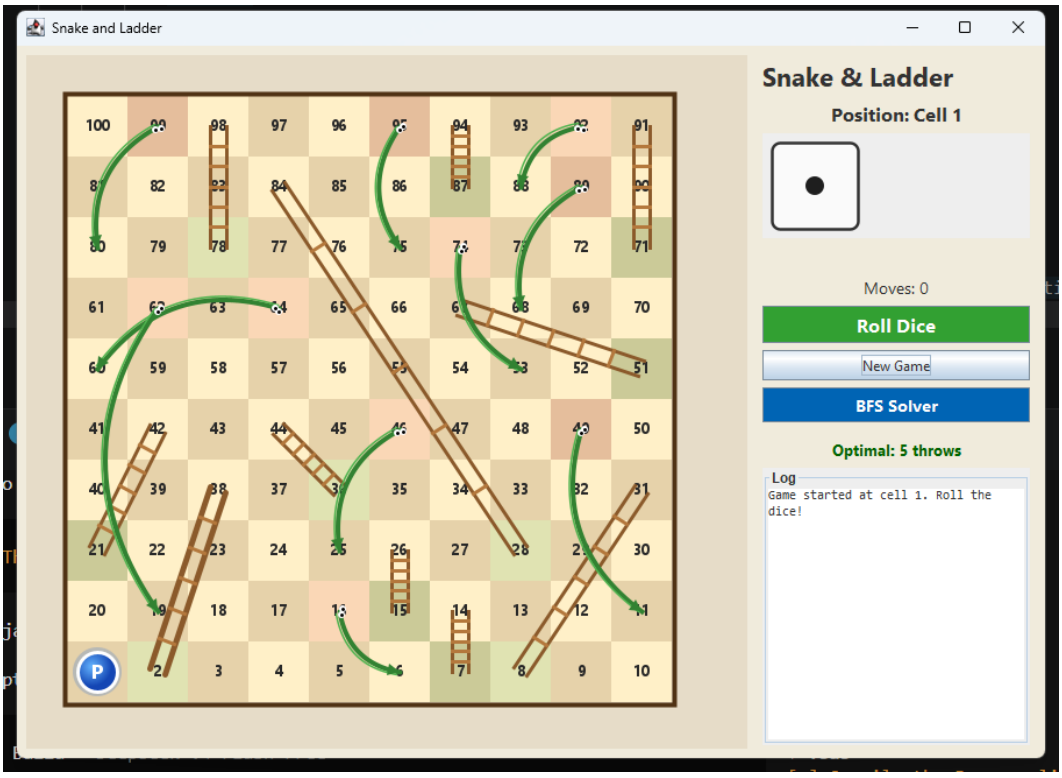


Figure 2: BFS Solver dialog with optimal path and step details.

7.3 BFS Path Highlighted on Board

The "Highlight on Board" button in the BFS dialog illuminates all cells on the optimal path with a gold/orange overlay. This provides a clear visual representation of the optimal route from cell 1 to cell 100.

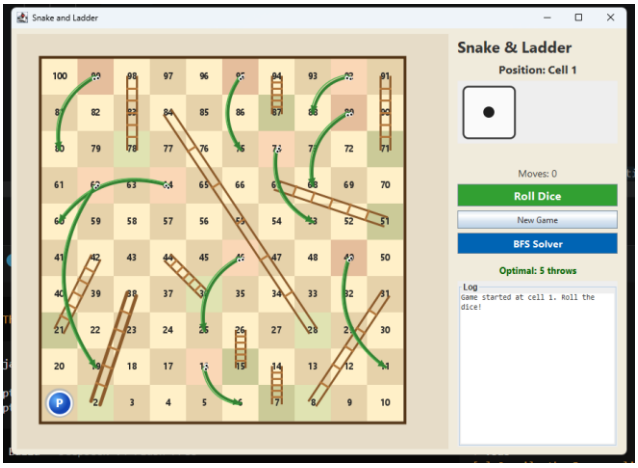


Figure 3: Board with the BFS optimal path highlighted in gold.

7.4 Mid-Game State

After several dice rolls, the player token has advanced through the board. The log panel records each roll and move, and the move counter tracks total throws used.

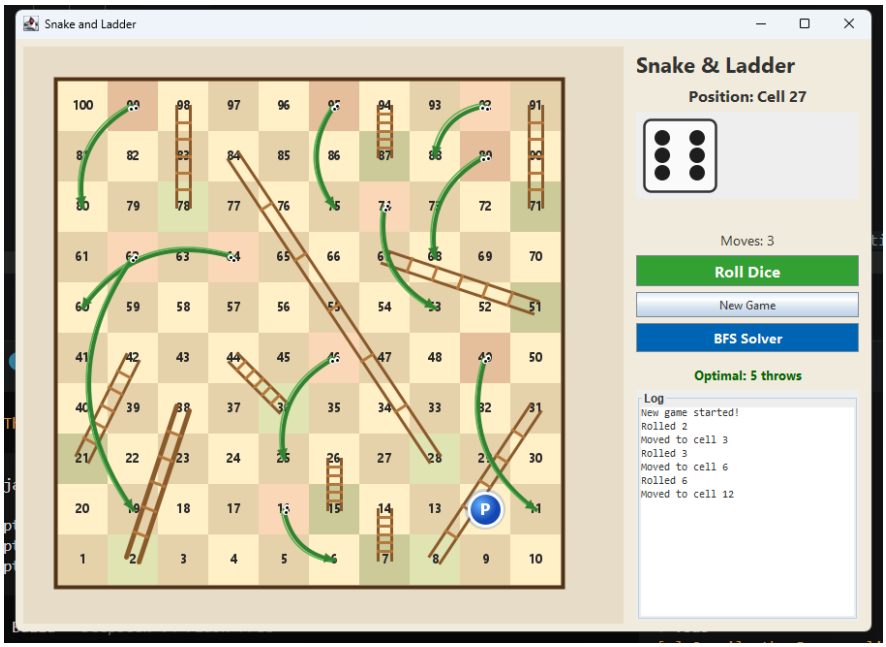


Figure 4: Mid-game state after several dice rolls.

8. Conclusion and Future Work

8.1 Conclusion

This project successfully delivers a fully interactive Snake and Ladder game with a polished graphical interface and an integrated BFS shortest-path solver. The implementation demonstrates several important computer science concepts:

- Graph theory — modeling a real-world game board as a directed graph
- BFS algorithm — finding the shortest path in an unweighted graph
- Java Swing — building responsive, event-driven GUI applications
- Custom rendering — using Java2D for vector graphics, gradients, and animations
- Concurrency — managing asynchronous animations on the Event Dispatch Thread

The BFS solver consistently finds the optimal path from cell 1 to cell 100 in 5 throws (with the default board configuration), completing in under 1 millisecond. The visual path highlighting and step-by-step dialog make the solver output accessible and educational.

8.2 Limitations

- Single-player only — no multiplayer or AI opponent.
- The board layout is hard-coded — no runtime configurability.
- No save/load functionality for game state.

- Sound effects and advanced animations are absent.
- The BFS solver does not handle stochastic (probabilistic) analysis.

8.3 Future Enhancements

- **Multi-player support:** Add 2–4 players with turn-based play, each with a distinct token color.
- **AI Opponent:** An AI that uses BFS to play optimally, providing a challenging single-player experience.
- **Configurable boards:** Load snake/ladder positions from a text file or JSON, allowing custom board designs.
- **Different dice:** Support 4-sided, 8-sided, or 20-sided dice with configurable ranges.
- **Statistics panel:** Track win rates, average throws, luck index, and other game statistics.
- **Sound effects:** Add sounds for dice rolling, ladder climbing, snake sliding, and victory.
- **Undo move:** Allow the player to undo the last roll.
- **Themes/skins:** Different visual themes (classic wood, modern, dark mode).
- **Network play:** Online multiplayer via sockets or REST API.
- **Mobile version:** Port to Android using a cross-platform framework.

In summary, this project provides a solid foundation for a Snake and Ladder game that is both fun to play and educational in its demonstration of graph algorithms. The clean separation of concerns, well-documented code, and comprehensive feature set make it an excellent reference implementation for anyone learning Java GUI programming or algorithm visualization.

Appendix A: Complete Source Code

The entire project is contained in a single Java source file (SnakeLadder.java, 678 lines). The complete source code is reproduced below for reference.

https://github.com/avijitsaha01/Design-and-Analysis-of-Algorithms